

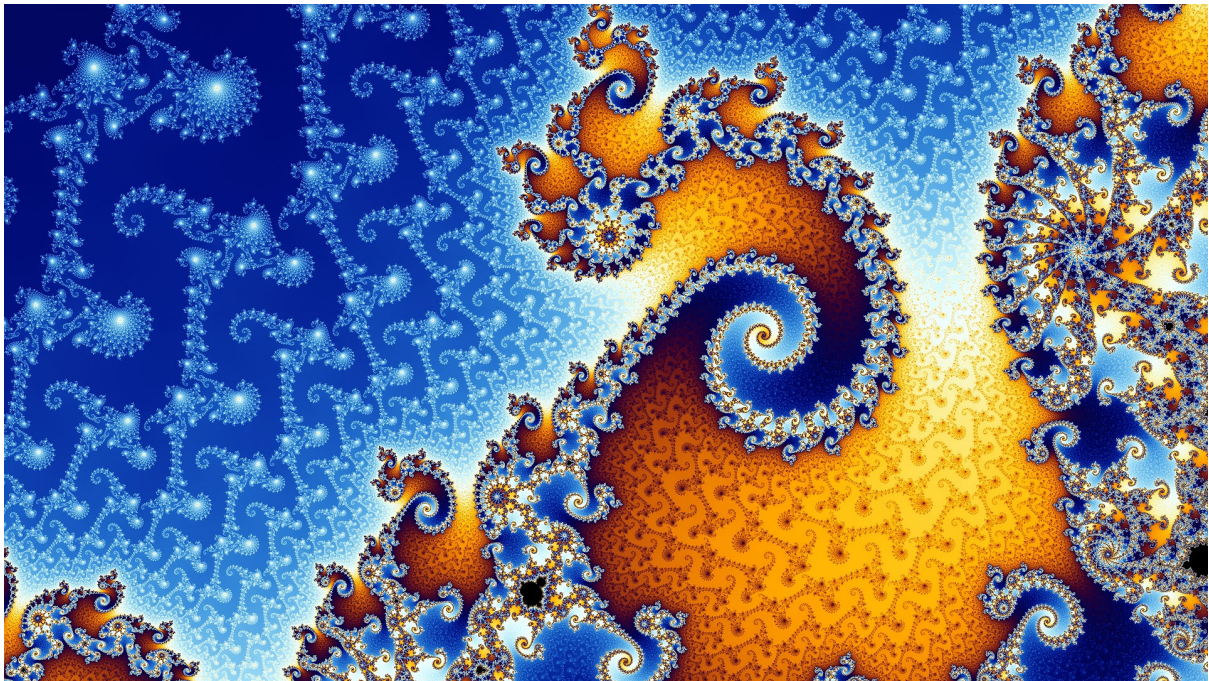
CIARLO Emilio Luca

CLARET Nathan

FABRE Lucas

ROUSSEL Hugo

Optimisation différentiable 2



2022-2023

M.El-Mahbouby - M.Couffignal

1	INTRODUCTION	2
2	Des fractales avec la méthode de Newton	3
2.1	Question 1	3
2.2	Question 2: Application fractale avec Pyhton	4
3	SVM avec noyau	16
3.1	Question 1	16
3.2	Question 2	17
3.3	Question 3	18
3.4	Question 4	20
3.5	Question 5	20
3.6	Question 6	24
3.7	Question 7	25
3.8	Question 8	26
4	Conclusion	27

L'optimisation différentiable est une branche de l'optimisation mathématique qui consiste à trouver le minimum (ou le maximum) d'une fonction à l'aide de méthodes reposant sur le calcul de dérivées. Il est largement utilisé dans de nombreux domaines, notamment l'apprentissage automatique, le traitement du signal et la finance pour résoudre des problèmes tels que la classification, la régression, la reconnaissance de formes, la prévision de séries chronologiques, la conception de circuits électroniques et la planification de portefeuille.

L'optimisation différentiable est basée sur l'idée que si une fonction est différentiable, elle peut être approchée par une fonction linéaire autour de son minimum (ou maximum). Cette approximation linéaire est obtenue en utilisant le gradient de la fonction, qui indique la direction de croissance maximale de la fonction. En utilisant ces informations, les méthodes d'optimisation différentiables cherchent à trouver le minimum (ou le maximum) de la fonction le long de la direction opposée du gradient.

Les méthodes d'optimisation différentiables sont particulièrement utiles lorsque la fonction à optimiser est complexe et que les solutions analytiques n'existent pas ou sont difficiles à trouver.

Dans une première partie nous allons tracer des fractales grâce à la méthode de Newton, ensuite dans la deuxième partie nous réaliser de l'apprentissage de chiffres manuscrit à l'aide de réseaux de neurones et de l'algorithme d'Uzawa.

CHAPTER 2

DES FRACTALES AVEC LA MÉTHODE DE NEWTON

Les fractales sont des objets mathématiques complexes qui présentent des propriétés fascinantes telles que l'auto-similarité à différentes échelles, des dimensions de Hausdorff non entières et une complexité infinie. Elles ont été découvertes en 1975 par Benoît Mandelbrot, qui a également développé une théorie mathématique de leur formation et de leur comportement.

La méthode de Newton est une méthode numérique utilisée pour trouver les racines d'équations complexes en itérant des formules mathématiques. Il peut être utilisé pour dessiner des fractales.

La première partie de notre travail consistera à créer des fractales à l'aide des formules de Newton.

2.1 Question 1

La méthode de Newton permet de construire des algorithmes permettant de résoudre des systèmes d'équations non-linéaires.

$$\begin{cases} x_0 \in \mathbb{R}^n \\ x_{k+1} = x_k - g'_k(x_k)^{-1}g(x) \end{cases}$$

Dans le cas de notre projet nous on a:

$$P(x) = (x-1)(x + \frac{1}{2} - a)(x + \frac{1}{2} + a)$$

l'algorithme de Newton s'écrit alors:

$$\begin{cases} x_0 \in \mathbb{R}^n \\ x_{k+1} = x_k - P'(x)^{-1}P(x) \end{cases}$$

Pour calculer la dérivé de P, nous il est convenable de simplifier son écriture avant tout.

$$P(x) = (x^2 + \frac{1}{2}x - ax - x - \frac{1}{2} + a)(x + \frac{1}{2} + a)$$

$$P(x) = x^3 + x^2 + \frac{1}{4}x - a^2x - x^2 - x - \frac{1}{4} + a^2$$

$$P(x) = x^3 + x(-\frac{3}{4} - a^2) - \frac{1}{4} + a^2$$

On obtient trivialement en dérivant:

$$P'(x) = 3x^2 - \frac{3}{4} - a^2$$

2.2 Question 2: Application fractale avec Python

Nous allons maintenant vous montrer toutes les fonctions que nous avons définies pour tracer des fractales à l'aide de l'algorithme de Newton.

On commence tout d'abord par définir une fonction "newton-R" qui reprend l'algorithme de Newton afin de trouver les racines du polynôme $P(x)$ de l'énoncé. Nous insérerons dans notre algorithme (point de départ) une matrice qu'on appellera "x0" et qui représentera le plan complexe avec sur les lignes, l'axe des imaginaires et sur les colonnes, l'axe des réels. Notre algorithme fonctionnera donc de manière matricielle en faisant les itérations sur chaque terme de notre matrice de départ. Les deux dernières boucles permettent de remplacer les termes qui n'ont pas convergé par des np.inf pour pouvoir tracer ces cas de non convergence. On définit alors qu'un terme ne converge pas si la "distance" entre le terme d'avant dernière itération et le terme de dernière itération est supérieur à la précision que l'on a choisie, nommée: ep.

Newton:

```
1 def newton_P(x0,a,ep,max_iter):
2     x=x0
3     compteur=0
4     for k in range(max_iter):
5         xpl = x - P(x,a) / dP(x,a)
6         if np.linalg.norm(xpl-x) < ep:
7             break
8         xm1=x
9         x=xpl
10        compteur += 1
11        print('iteration_Newton: {}'.format(compteur))
12
13    if isinstance(x, np.ndarray):
14        for i in range(np.shape(x)[0]):
15            for j in range(np.shape(x)[1]):
16                if np.abs(x[i,j]-xm1[i,j])>ep:
17                    x[i,j]=np.inf
18    return x
```

Le polynôme et sa dérivée utilisés pour cet algorithme sont :

Fonctions polynômes:

```
1 def P(x,a):
2     return (x-1) * (x+0.5-a) * (x+0.5+a)
3
4 def dP(x,a):
5     return (3*x**2) - (3/4) - (a**2)
```

Afin de tracer, avec l'algorithme de Newton, la fractale associée à ce polynôme, nous avons défini la fonction "fractale-vid-P1". En effet, cette fonction crée dans un premier temps le plan complexe sous forme de matrice qui nous servira de point de départ pour l'algorithme de Newton. Nous allons récupérer le résultat de l'algorithme et regarder de quelle racine les termes de la matrice sont le plus proche pour les associer chacun à une couleur en fonction du terme qui se rapproche le plus de la première racine, de la deuxième ou de la troisième. Nous associons également à une couleur le dernier cas qui est le cas de non convergence. Les termes qui ne convergent pas sont marqués dans la matrice que renvoie Newton par un "np.inf".

La première version de notre fonction utilisait une double boucle pour parcourir tous les termes un par un. Nous avons réussi à réaliser toutes ces opérations matriciellement, améliorant grandement la rapidité de notre code.

On répète tout cela n fois en réduisant à chaque fois le domaine du plan complexe en le centrant sur un point qu'on a choisi ("zoom-point") afin de faire un zoom en un point précis de la fractale :

Fractale:

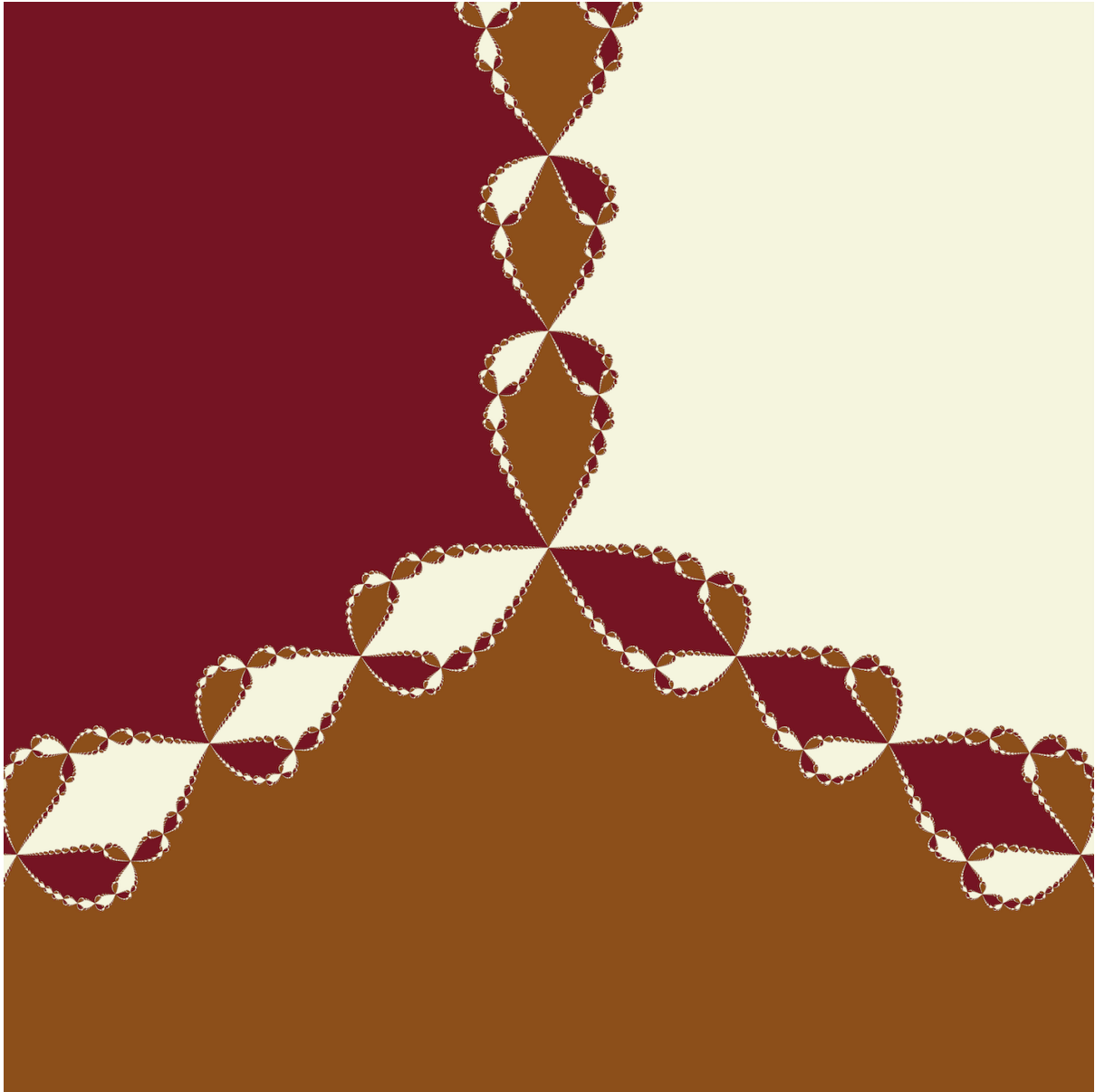
```

1 def fractale_vid_P1(n, a, nbiter, xmin=-2, xmax=2, zoom_point=(0, 0)):
2
3     r1, r2, r3 = 1, -1/2+a, -1/2-a
4     ep = 0.01
5     etape = []
6     fractal = np.zeros((n, n, 3))
7
8     for k in range(1, nbiter+1):
9         zoom_xmin = (1/k) * (xmin - zoom_point[0]) + zoom_point[0]
10        zoom_xmax = (1/k) * (xmax - zoom_point[0]) + zoom_point[0]
11
12        zoom_ymin = (1/k) * (xmin - zoom_point[1]) + zoom_point[1]
13        zoom_ymax = (1/k) * (xmax - zoom_point[1]) + zoom_point[1]
14
15
16
17        Re = np.linspace(zoom_xmin, zoom_xmax, n).reshape(-1, 1)
18        Im = np.linspace(zoom_ymin * 1j, zoom_ymax * 1j, n).reshape(-1, 1)
19        x0 = np.outer(Re, np.ones(n)) + np.outer(np.ones(n), Im)
20
21        x1 = newton_P(x0, a, ep, 50)
22
23        distances = np.abs(np.dstack((x1 - r1, x1 - r2, x1 - r3)))
24        fractal[Rplusproche == 0] = [150, 75, 0]
25        fractal[Rplusproche == 1] = [245, 245, 220]
26        fractal[Rplusproche == 2] = [128, 0, 32]
27        etape.append(np.uint8(fractal.copy()))
28    imageio.mimsave('fractale.gif', etape, fps=20, duration=0.5)

```

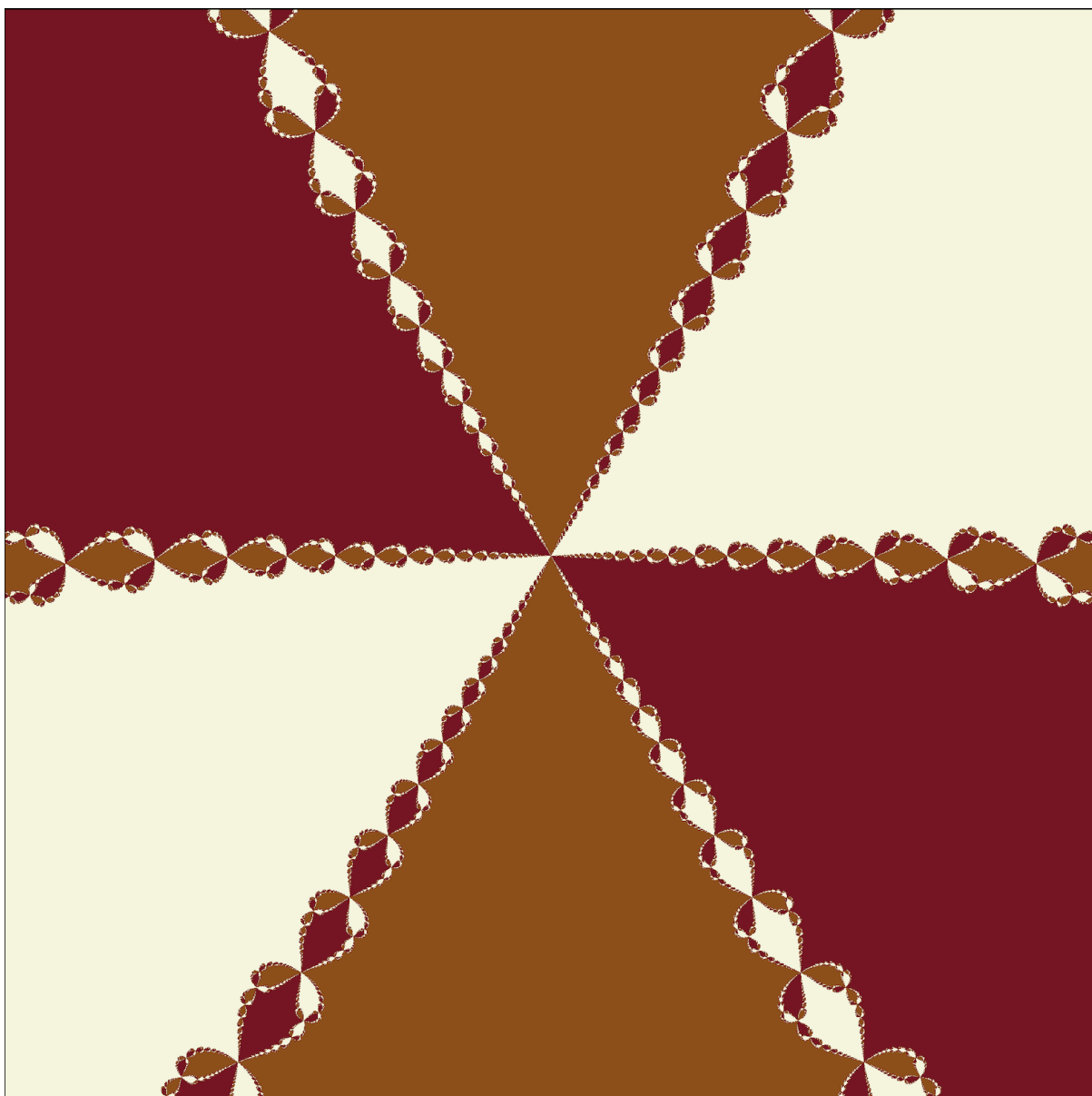
Test avec la fractale de Newton :

Avec une résolution de 2000 pixels ($n=2000$), un nombre complexe $a = \frac{\sqrt{3}}{2}i$ et $x_{\min}=-2$ et $x_{\max}=2$, on obtient bien la fractale de newton :



On peut observer que l'ensemble des points du plan complexe ont convergé car aucun pixels de la fractale sont de couleur noir.

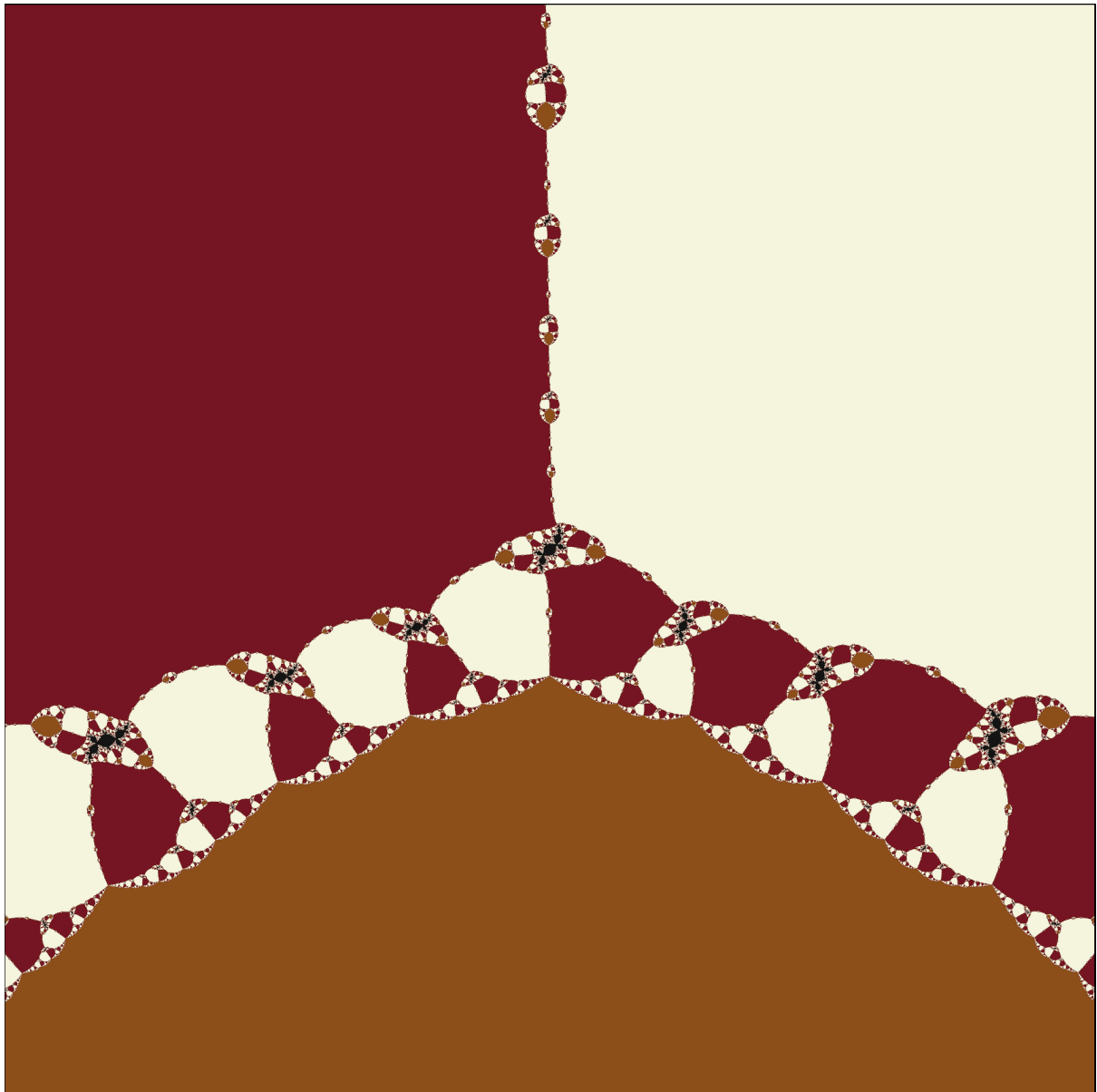
Après un zoom de 15 itérations au centre de la fractale (point $(0,0)$), on obtient la fractale zoomée suivante :



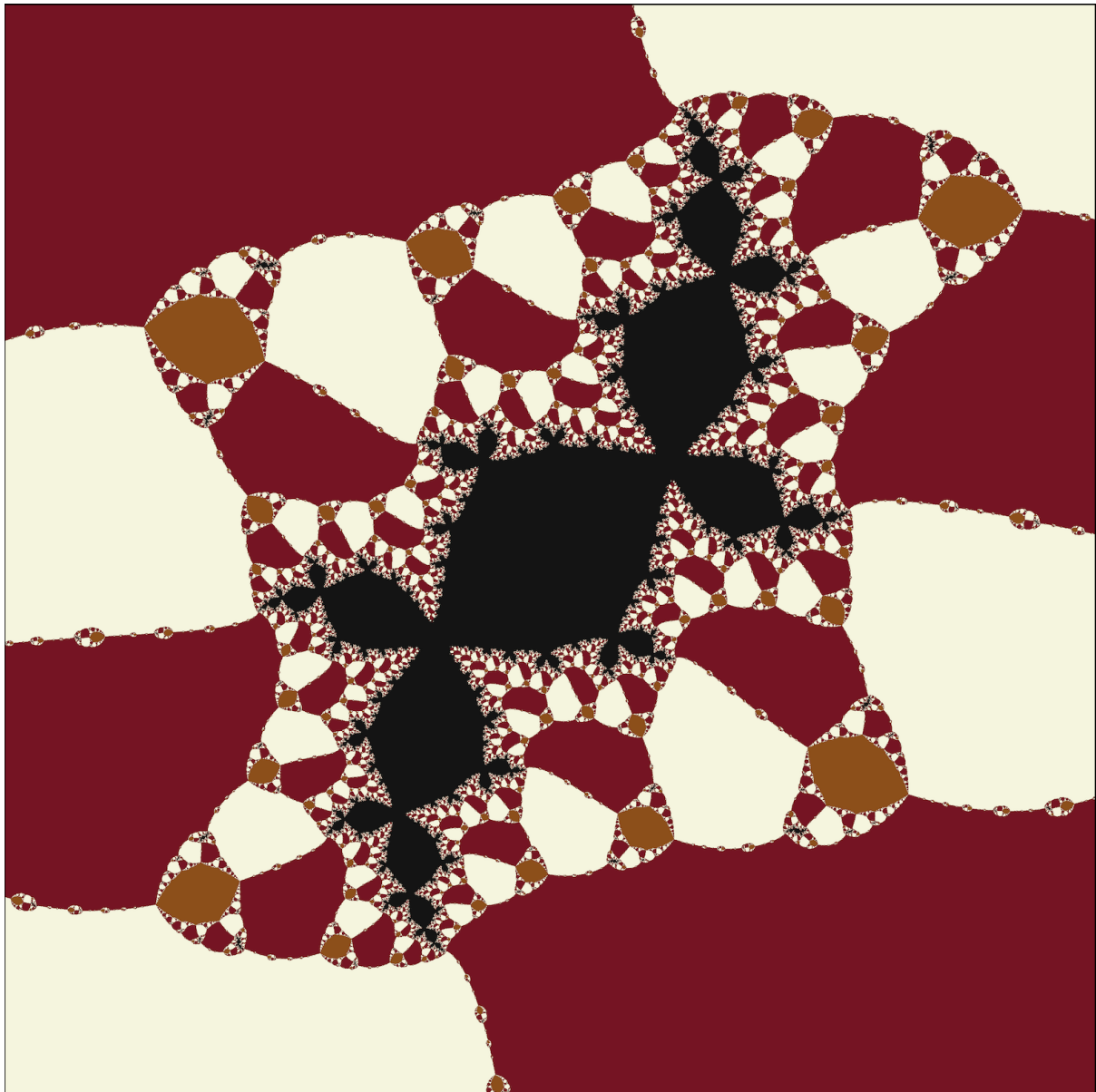
On observe également que l'ensemble des pixels convergent sur cette photo (il n'y a pas de pixels noirs).

Test pour le lapin de Douady :

En changeant uniquement la valeur du nombre complexe a : $a = -0,00508 + 0,33136i$, on peut observer la présence d'un lapin de Douady. En effet avec une résolution de 3000 pixels, on obtient la fractale suivante :



On peut déjà observer que certaine région de point de converge pas. Zoomons au centre de la figure autour du point $(0,0)$:



On voit bien l'apparition du lapin de Douady. Ce "lapin" est représenté par des pixel noirs (donc qui convergent pas) au centre de la figure.

Fractale du polynome $P2(z, a) = z^6 + z^3 - 1$

Nous allons maintenant tracer la fractale associée a un polynome trouvé sur internet : $P2(z, a) = z^6 + z^3 - 1$

Fonctions pôlynomes P2:

```

1
2 def P2(z,a):
3     return z**6 + z**3 -1
4
5
6 def dP2(z,a):
7     return 6*z**5 + 3*z**2

```

L'algorithme de Newton est le même à la seul différence qu'on utilise le polynome P2 et sa dérivé :

Fonction Newton pour le polynôme P2:

```

1 def newton_P2(x0,a,ep,max_iter):
2     x=x0
3     compteur=0
4     for k in range(max_iter):
5         xp1 = x - P2(x,a) / dP2(x,a)
6         if np.linalg.norm(xp1-x) < ep:
7             break
8         xm1=x
9         x=xp1
10        compteur += 1
11        print('it ration_Newton:{}'.format(compteur))
12
13
14
15    if isinstance(x, np.ndarray):
16        for i in range(np.shape(x)[0]):
17            for j in range(np.shape(x)[1]):
18                if np.abs(x[i,j]-xm1[i,j])>ep:
19                    x[i,j]=np.inf
20
21    return x

```

La fonction pour tracer la fractale est la même que la précédente sauf que dans celle-ci, on définit une liste "coef" qui contient les coefficients du polynôme P2. On utilise ensuite la fonction roots de numpy pour calculer les racines du polynôme :

Fonction fractale du polynôme P2:

```

1
2 def fractale_vid_P2(n, a, nbiter, xmin=-2, xmax=2, zoom_point=(0, 0)):
3     ep = 0.01
4     etape = []
5
6     coef=[1,0,0,1,0,0,-1]
7     racine=np.roots(coef)
8
9     fractal = np.zeros((n, n, 3))
10
11    for k in range(1, nbiter+1):
12        zoom_xmin = (1/k) * (xmin - zoom_point[0]) + zoom_point[0]
13        zoom_xmax = (1/k) * (xmax - zoom_point[0]) + zoom_point[0]
14
15        zoom_ymin = (1/k) * (xmin - zoom_point[1]) + zoom_point[1]
16        zoom_ymax = (1/k) * (xmax - zoom_point[1]) + zoom_point[1]
17
18        Re = np.linspace(zoom_xmin, zoom_xmax, n).reshape(-1, 1)
19        Im = np.linspace(zoom_ymin * 1j, zoom_ymax * 1j, n).reshape(-1, 1)
20        x0 = np.outer(Re, np.ones(n)) + np.outer(np.ones(n), Im)
21
22        x1 = newton_P2(x0, a, ep, 50)
23
24        distances = np.abs(np.dstack((x1 - racine[0], x1 - racine[1], x1 -
25            racine[2], x1 - racine[3], x1 - racine[4], x1 - racine[5])))
26        Rplusproche = np.argmin(distances, axis=2)
27
28        fractal[Rplusproche == 0] = [150, 75, 0]
29        fractal[Rplusproche == 1] = [245, 245, 220]
30        fractal[Rplusproche == 2] = [128, 0, 32]
31        fractal[Rplusproche == 3] = [128, 179, 32]
32        fractal[Rplusproche == 4] = [12, 18, 210]
33        fractal[Rplusproche == 5] = [0, 199, 153]
34        fractal[x1 == np.inf] = [20, 20, 20]

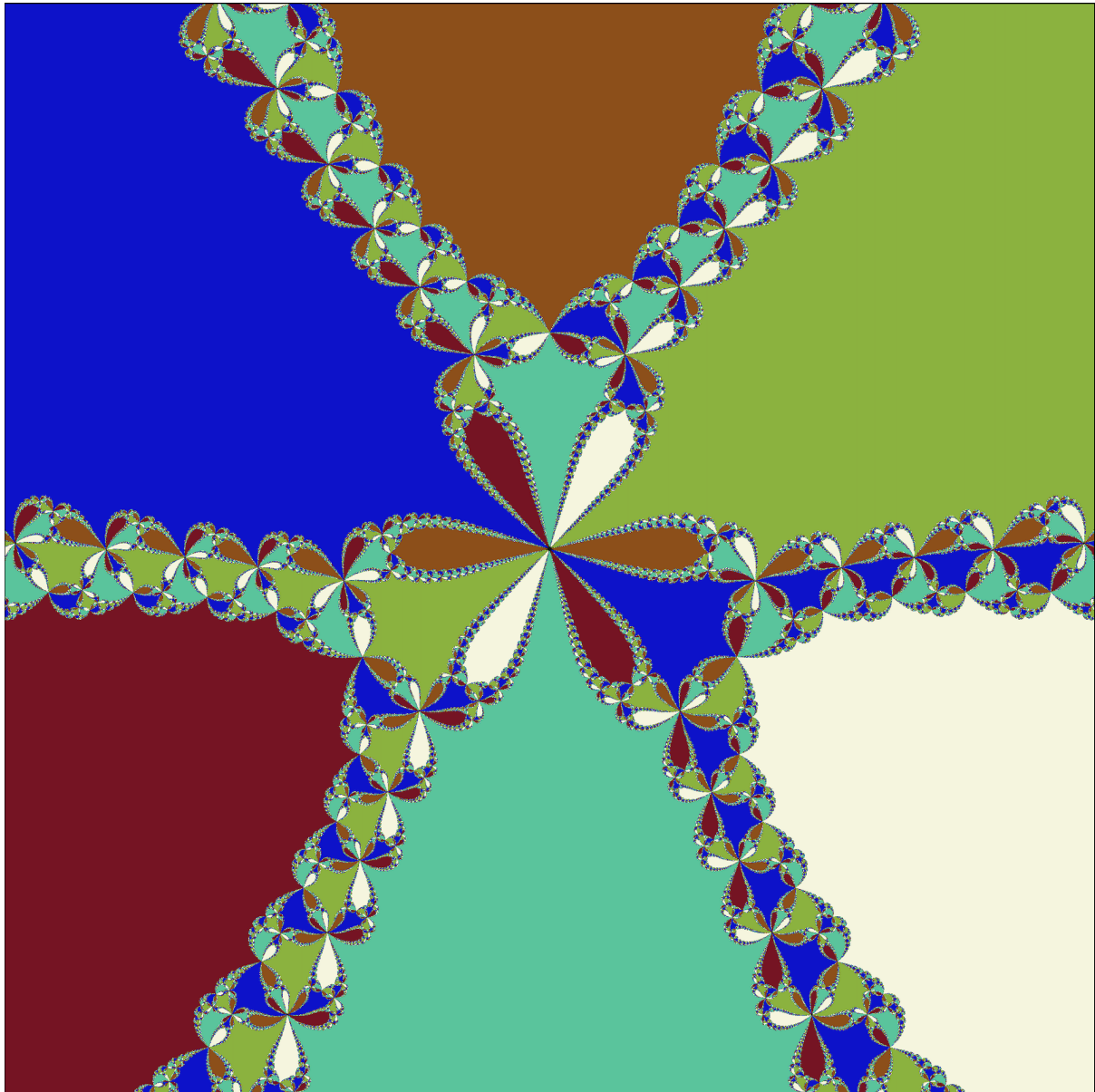
```

```

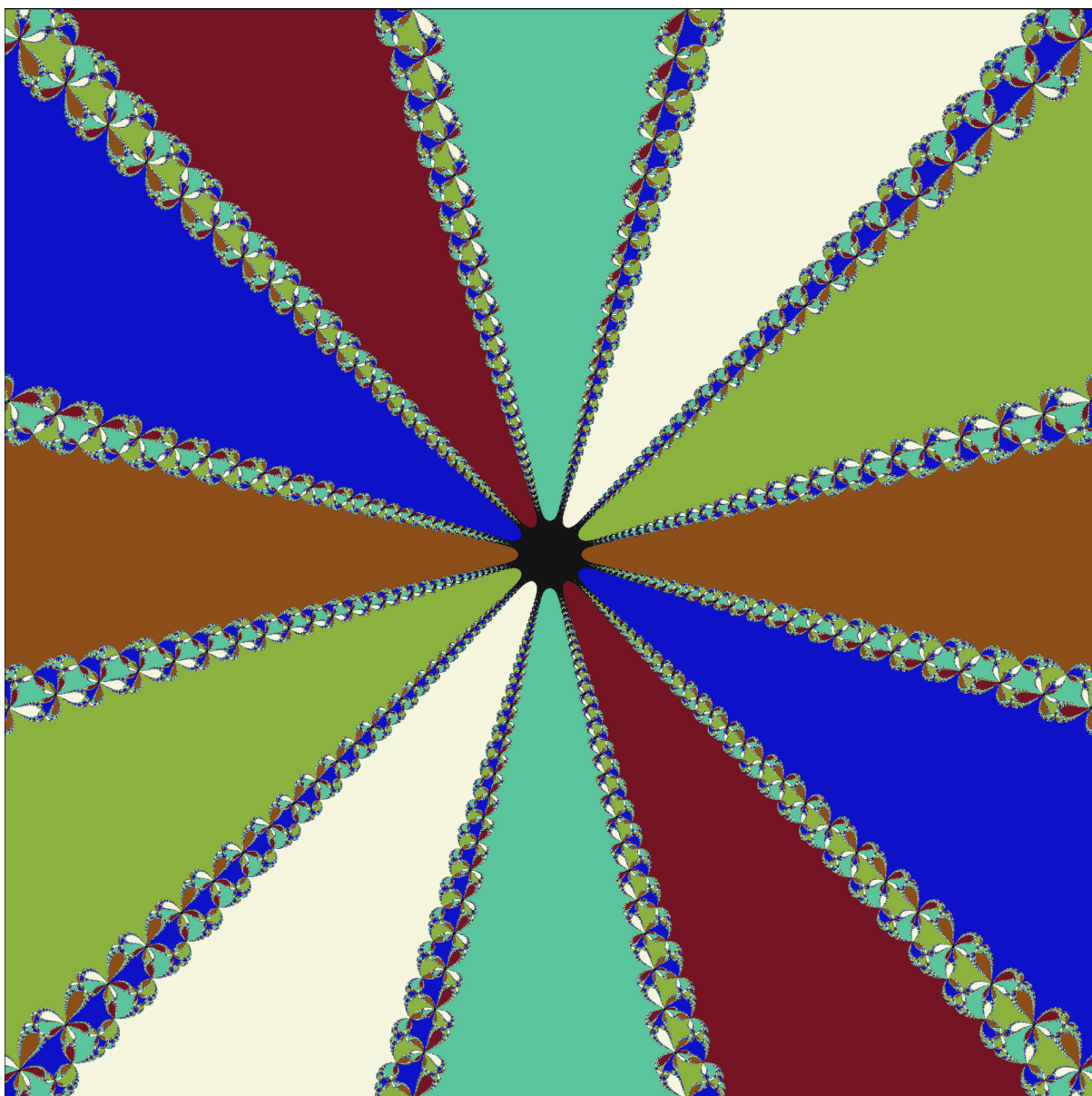
35     etape.append(np.uint8(fractal.copy()))
36
37     imageio.mimsave('fractale.gif', etape, fps=20, duration=0.5)

```

Avec une résolution de 2000 pixels, un nombre complexe $a = 0,3 + 0,5i$ et un point de départ $x_{\min}=-2$ et $x_{\max}=2$, on obtient la magnifique fractale suivante où les 7 différents cas de convergences sont représenté par les 7 couleurs présentes sur la fractale.



Ensuite nous zoomons au centre de cette fractale (point 0,0) pour vérifier sa converge ou non convergence. On obtient ainsi l'image suivante:



On peut voir sur ce zoom qu'il y a des pixels qui ne convergent pas (représentés en noir au centre de l'image).

Fractale de Mandelbrot :

Nous allons implémenter maintenant l'algorithme de Mandelbrot pour nous permettre de tracer les fractales associés à cette algorithme et de ne plus être limité à celui de Newton. L'algorithme de Mandelbrot est défini de la manière suivante :

$$\begin{cases} z_0 = 0 \\ z_{n+1} = z_n^2 + c \end{cases}$$

La fonction "mandelbrot" nous permet d'appliquer l'algorithme de Mandelbrot à une matrice initiale x_0 qui est le plan complexe :

Fonction Mandelbrot:

```
1
2 def mandelbrot(x0,c,max_iter):
3     x=x0
4     compteur=0
5     for k in range(max_iter):
6         xp1 = np.square(x)+ c
7         x=xp1
8         print('iteration_{}_algorithme_{}_mandelbrot_{}:{}'.format(compteur))
9         compteur+=1
10    return x
```

La fonction qui permet de tracer les fractales de Mandelbrot est exactement la même que la fonction précédente : "fractale-vid-P2" à la seule différence qu'on applique l'algorithme de Mandelbrot au plan complexe :

Fonction fractale Mandelbrot:

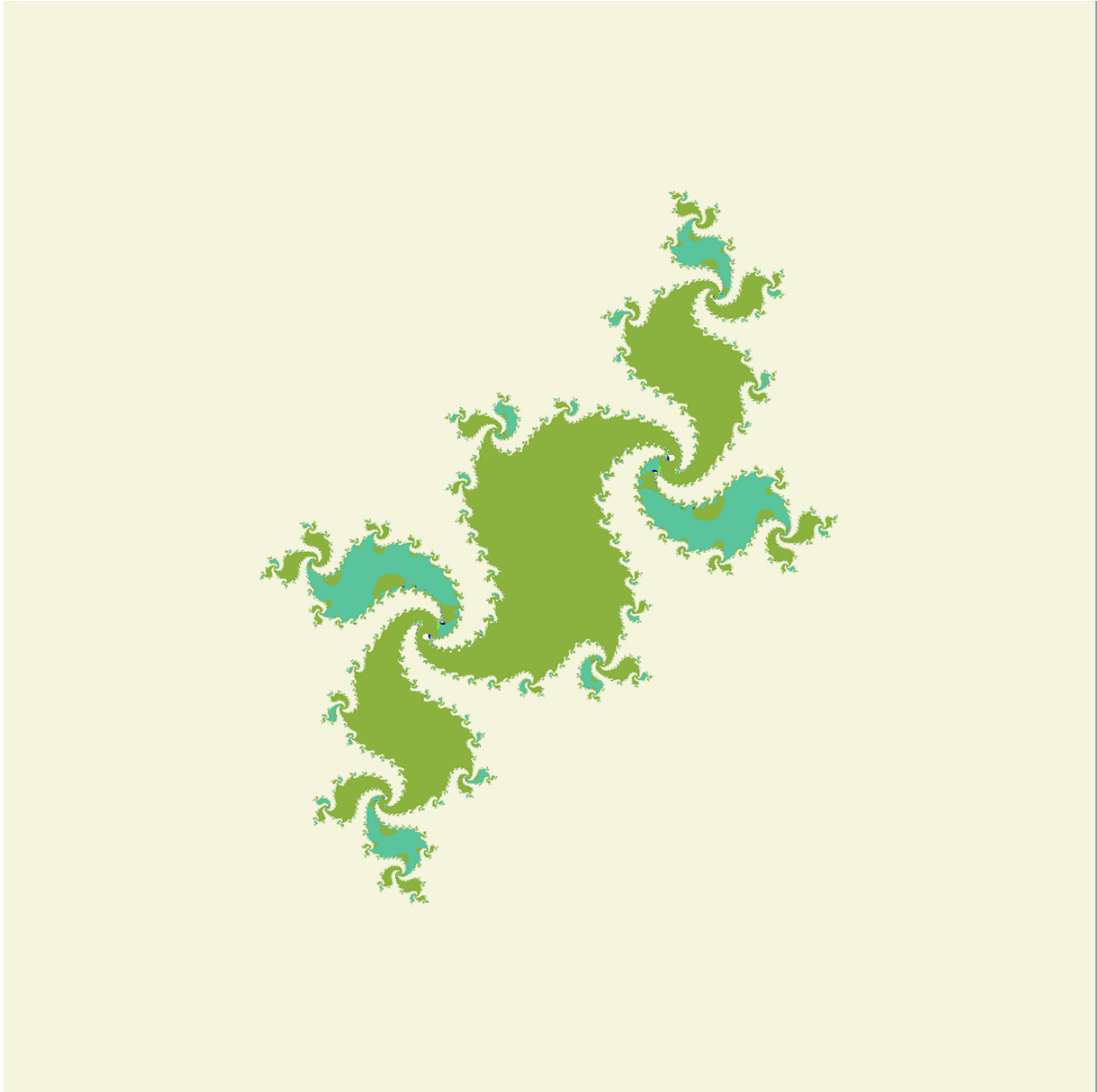
```
1
2 def fractale_vid_mandelbrot(n, a, nbiter, xmin=-2, xmax=2, zoom_point=(0, 0)):
3     etape = []
4
5     coef=[1,0,0,1,0,0,-1]
6     racine=np.roots(coef)
7
8     fractal = np.zeros((n, n, 3))
9
10    for k in range(1, nbiter+1):
11        zoom_xmin = (1/k) * (xmin - zoom_point[0]) + zoom_point[0]
12        zoom_xmax = (1/k) * (xmax - zoom_point[0]) + zoom_point[0]
13
14        zoom_ymin = (1/k) * (xmin - zoom_point[1]) + zoom_point[1]
15        zoom_ymax = (1/k) * (xmax - zoom_point[1]) + zoom_point[1]
16
17        Re = np.linspace(zoom_xmin, zoom_xmax, n).reshape(-1, 1)
18        Im = np.linspace(zoom_ymin * 1j, zoom_ymax * 1j, n).reshape(-1, 1)
19        x0 = np.outer(Re, np.ones(n)) + np.outer(np.ones(n), Im)
20
21        x1 = mandelbrot(x0, a, 50)
22
23        distances = np.abs(np.dstack((x1 - racine[0], x1 - racine[1], x1 -
24            racine[2], x1 - racine[3], x1 - racine[4], x1 - racine[5])))
25        Rplusproche = np.argmin(distances, axis=2)
26
27        fractal[Rplusproche == 0] = [245, 245, 220]
28        fractal[Rplusproche == 1] = [0, 245, 20]
29        fractal[Rplusproche == 2] = [128, 0, 32]
30        fractal[Rplusproche == 3] = [128, 179, 32]
31        fractal[Rplusproche == 4] = [12, 18, 210]
32        fractal[Rplusproche == 5] = [0, 199, 153]
```

```

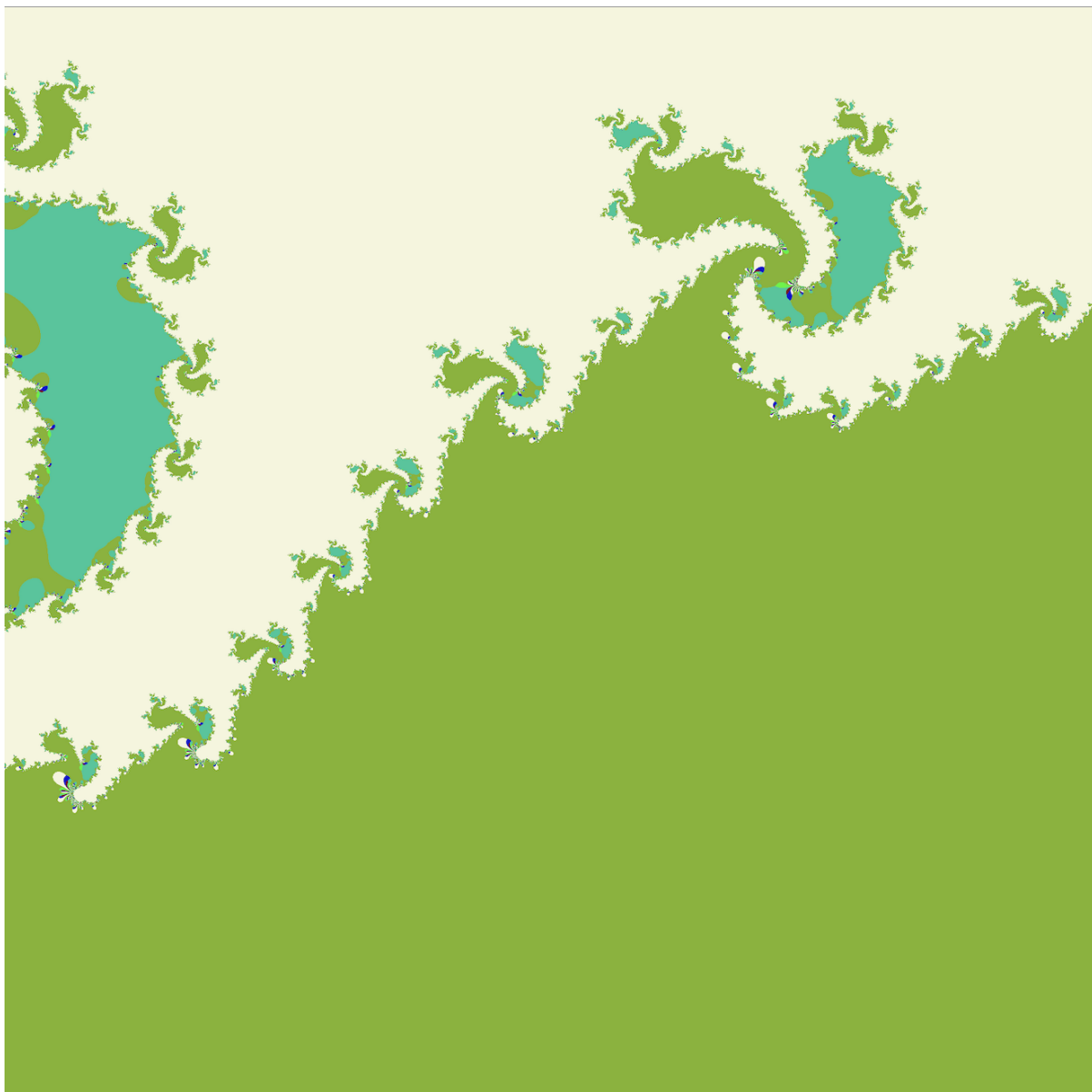
32     fractal[x1 == np.inf] = [20, 20, 20]
33
34     etape.append(np.uint8(fractal.copy()))
35
36     imageio.mimsave('fractale.gif', etape, fps=20, duration=0.5)

```

On peut maintenant tracer une fractale de Mandelbrot. Avec un nombre complexe $a = -0.1956 + 0.6958i$, une résolution de 2000 pixels et un point de départ à $x_{\min}=-2$ et $x_{\max}=2$, on obtient la magnifique fractale suivante :



On peut observer que tout les points du plan complexe converge.
Zoomons maintenant sur le point $(-0,45,0)$:



On observe encore une fois que tout les points convergent.

Une machine à vecteurs de support du noyau plus communément appelée SVM est une extension de la SVM classique qui permet la modélisation de problèmes non linéaires en transformant les données d'entrée dans un espace de dimension supérieure. Cette transformation est effectuée par une fonction noyau, qui mesure la similarité entre deux points dans l'espace d'entrée et calcule leur produit scalaire dans l'espace transformé.

L'idée clé derrière Kernel SVM est de trouver un hyperplan séparateur qui maximise la marge entre deux points de classes différentes dans l'espace transformé. Cette marge est définie comme la distance entre l'hyperplan séparateur et le point le plus proche de chaque classe, également appelé vecteur support. Les SVM du noyau sont donc capables de construire des frontières de décision non linéaires à l'aide de fonctions de noyau telles que les noyaux gaussiens, les noyaux polynomiaux ou les noyaux sigmoïdes.

Les SVM du noyau sont souvent utilisées pour les problèmes de classification, mais peuvent également être utilisées pour les problèmes de régression. Ils sont populaires dans l'apprentissage automatique en raison de leur capacité à modéliser des données non linéaires tout en conservant des formulations mathématiques simples. Cependant, leur utilisation peut être coûteuse en calcul, en particulier pour les grands ensembles de données.

3.1 Question 1

Dans cette première question, nous allons débiter par le cas de la classification binaire de l'apprentissage de deux catégories. Nous rappelons ici un problème de classification binaire : supposons donné un ensemble fini E de taille $N \in \mathbb{N}^*$ partitionné en deux sous-ensembles disjoints de points de $\mathbb{R}^n$:

$$E_1 = \{u_i \in \mathbb{R}^n \mid i \in [1, p]\}$$

$$E_2 = \{v_j \in \mathbb{R}^n \mid j \in [1, q]\}$$

où p et q sont des entiers non nuls. On rappelle que ces données forment ce que l'on appelle les données d'entraînements. Nous nous placerons dans le formalisme des machines à vecteurs supports (SVM).

On rappelle que dans le cas linéaire le problème revient à résoudre :

$$(P) : \begin{cases} \min \frac{1}{2} \|w\|^2 \\ \text{Sous les contraintes : } \begin{cases} w^T u_i - b \geq 1 & \forall i \in [1, p] \\ -w^T v_j + b \geq 1 & \forall j \in [1, q] \end{cases} \end{cases}$$

De plus nous avons une fonction noyau telle que: $\phi : E \rightarrow H$ avec H un espace vectoriel et un produit scalaire $\langle \bullet, \bullet \rangle$ sur H tel que:

$$k(x, y) = \langle \phi(x), \phi(y) \rangle$$

De plus, tout vecteur $w \in H$ se décompose d'après le théorème de Mercer sous la forme:

$$w = \sum_{i=1}^N a_i \phi(x_i)$$

avec $x_i \in E$ et $N = p + q$.

En utilisant la fonction de redescription ϕ on peut écrire une forme similaire au problème (P) de la partie dans l'espace H donné sous la forme suivante :

$$(P_H) : \begin{cases} \min \frac{1}{2} \langle w, w \rangle \\ \text{Sous les contraintes : } \begin{cases} \langle w, \phi(u_i) \rangle - b \geq 1 & \forall i \in [1, p] \\ -\langle w, \phi(v_j) \rangle + b \geq 1 & \forall j \in [1, q] \end{cases} \end{cases}$$

Ainsi avec les contraintes du problème (P_H) on a:

$$\begin{cases} \langle w, \phi(u_i) \rangle - b \geq 1 \\ -\langle w, \phi(v_j) \rangle + b \geq 1 \end{cases}$$

Où l'on rappelle que : $w = \sum_{i=1}^N a_i \phi(x_i)$

Ainsi on a :

$$\Leftrightarrow \begin{cases} \langle \sum_{i=1}^N a_i \phi(x_i), \phi(u_i) \rangle - b \geq 1 \\ -\langle \sum_{i=1}^N a_i \phi(x_i), \phi(v_j) \rangle + b \geq 1 \end{cases} \Leftrightarrow \begin{cases} \sum_{i=1}^N a_i \langle \phi(x_i), \phi(u_i) \rangle - b \geq 1 \\ -\sum_{i=1}^N a_i \langle \phi(x_i), \phi(v_j) \rangle + b \geq 1 \end{cases}$$

De plus nous savons que nous pouvons exprimer la fonction noyau telle que: $k(x, y) = \langle \phi(x), \phi(y) \rangle$
Ainsi:

$$\Leftrightarrow \begin{cases} \sum_{i=1}^N a_i k(x_i, u_i) - b \geq 1 \\ -\sum_{i=1}^N a_i k(x_i, v_j) + b \geq 1 \end{cases}$$

On pose $K_{i,i} = k(x_i, u_i)$ et $K_{i,j} = k(x_i, v_j)$ qui correspondent aux fonctions noyaux. On peut également exprimer $\sum_{i=1}^N a_i$ sous forme matricielle A , on a ainsi:

$$\Leftrightarrow \begin{cases} AK_{i,i} - b \geq 1 \\ -AK_{i,i} + b \geq 1 \end{cases} \Leftrightarrow \begin{cases} 1 - AK_{i,i} + b \leq 0 \\ 1 + AK_{i,i} - b \leq 0 \end{cases}$$

On obtient finalement la fonction noyau ainsi que la matrice A et le vecteur b .

3.2 Question 2

Dans cette deuxième question, nous cherchons à écrire l'algorithme d'Uzawa appliqué à la résolution du problème de classification binaire. Ainsi nous pouvons poser d'après le cours:

$$\alpha 1_{p+q} - \mu_1 - \mu_2 = 0_{p+q} \Leftrightarrow \mu_{1,i} + \mu_{2,i} = \alpha$$

$\forall i \in [1, p + q]$ avec les $\mu_{1,i} \leq 0$ et $\mu_{2,i} \leq 0$ par hypothèse, ce qui implique que pour tout $i \in [1, p + q]$:

$$0 \leq \mu_{1,i} \leq \alpha$$

D'où on obtient l'algorithme d'Uzawa suivant pour résoudre le problème à marge souple tel que:

$$\begin{cases} \mu_0 \in (\mathbb{R}^+)^{p+q} \quad \rho > 0 \quad (\text{à choisir pour avoir la convergence}) \\ \begin{cases} x_k = -C^T \mu_k \\ \mu_{k+1} = p_{R_\alpha}(\mu_k + \rho(Cx_k + 1_{p+q})) \end{cases} \end{cases}$$

Où $R_\alpha = \{y \in \mathbb{R}^{p+q} \mid 0 \leq y_i \leq \alpha \quad \forall i \in [1, p+q]\}$

On rappelle que pour $y \in \mathbb{R}^{p+q}$ les composantes de $p_{R_\alpha}(y)$ sont : $p_{R_\alpha}(y)_i = \min(\max(0, y_i), \alpha)$ $\forall i \in [1, p+q]$.

Ainsi nous avons bien réécrit l'algorithme d'Uzawa appliqué à la résolution du problème de classification binaire.

3.3 Question 3

L'objectif de cette question est de détecter le chiffre 0 parmi les autres chiffres contenus dans la base de données MNIST. Nous utiliserons pour cela les 1000 premières données de la base de données mnist-train.csv et le noyau polynomial de degré 2 : $k(x, y) := (c + x^T y)^2$.

Afin de rédiger le programme pour l'algorithme d'Uzawa appliqué à la résolution du problème de classification binaire. Nous devons dans un premier temps écrire la fonction noyau que nous allons utiliser. Dans ce premier temps, nous utiliserons le noyau polynomial.

Noyau polynomial

```
1 def kernpol(x, y, c=105):
2     return (c+x.T@y)**2
```

Par la suite nous devons créer la matrice de contrainte, que l'on nommera C. Ainsi pour cela dans un premier temps, nous créons une fonction mat Gram qui aidera à créer notre matrice C plus rapidement.

Création de la matrice C

```
1 def mat_Gram(kern, x, y):
2     p=np.shape(x)[0]
3     q=np.shape(y)[0]
4     K=np.zeros((p,q))
5
6     for i in range(p):
7         for j in range(q):
8             K[i,j]=kern(x[i],y[j])
9     return K
10
11 def mat_C(kern, u, v, x):
12     p=np.shape(u)[0]
13     q=np.shape(v)[0]
14
15     return np.block([[mat_Gram(kern, u, x), np.ones((p,1))],
16                      [mat_Gram(kern, v, x), -np.ones((q,1))]])
```

Puis pour continuer dans notre lancée, nous nous devons de créer les matrices mu et w, mu nous servira à créer b qui sera l'ordonnée à l'origine de la droite qui séparera nos données. En sachant que w, représente les coefficient directeur de cette droite.

Création de mu et w

```
1
2 def mat_mu_w (C, epsilon=10**(-2), maxiter = 5000):
3
4     mu=np.ones(( np.shape(C)[0] ,1) )
5
```

```

6   cvmax =2 /( np. linalg . norm (C, ord =2) **2)
7   rho = cvmax /2
8
9   mu0=mu+epsilon
10  nb=0
11  while np.linalg.norm(mu-mu0) > epsilon and nb < maxiter :
12      mu0=mu
13      x=( -1) *(C.T @ mu)
14      mu= proj (mu+ rho *( C@x +np. ones (( np.shape(C)[0] ,1) ) ) )
15      nb+=1
16  x=( -1) *(C.T @ mu)
17  return mu,x[:-1,[0]]

```

Maintenant, on va créer le coefficient représentant l'ordonnée à l'origine de notre courbe séparant les deux jeux de données.

Calcul du coefficient b

```

1  def b(w,mu,p,C):
2
3      mu1 =mu [:p]
4      mu2 =mu[p:]
5      result1 =np. nonzero (mu1 ) [0]
6      result2 =np. nonzero (mu2 ) [0]
7      b=0
8      if result1.size >0 :
9          for i in np. nditer ( result1 ) :
10              b =(1/(2* len ( result1 ) ) ) *C[i ,:-1] @w +b
11
12      if result2.size >0 :
13          for j in np. nditer ( result2 ) :
14              b =(1/(2* len ( result2 ) ) ) *C[p+j ,:-1] @w+b
15      return b

```

Ainsi, maintenant que nous avons créé toutes nos données, nous permettant d'écrire l'algorithme d'UZAWA, précédemment rédigé, nous pouvons créer la fonction représentant cet algorithme . Ainsi celle-ci ressort w et b, lorsque nous rentrons le chiffre que nous voulons différencier des autres, le noyau que l'on choisit et les données d'entraînement.

Création de la fonction représentant l'algorithme d'UZAWA

```

1
2  def Uzawak(i,kern, imgs, labels):
3
4      nombredetection =i
5
6      indiceu =np. where ( labels == nombredetection )
7      indicev =np. where ( labels != nombredetection )
8      u= imgs [ indiceu ]
9      v= imgs [ indicev ]
10
11      p=len(u)
12
13      C=mat_C(kern,u,v,imgs)
14      mu,W=mat_mu_w(C)
15
16      B=b(W, mu, p, C)
17
18      return W, B

```

Afin de tester notre programme, nous nous devons d'implémenter une fonction $f(x)$ nous ressortant une valeur pouvant être inférieure ou supérieure à 0. Lorsque la valeur de sortie est inférieure à 0, alors cela veut dire que le chiffre pour lequel, on a testé n'est pas le même que le chiffre pour lequel nous avons entraîné w et b . Si la valeur est supérieure à 0, cela veut dire que le chiffre testé est le même pour lequel, on a entraîné notre programme.

Fonction f

```

1 def fk(nligne, imgs, w, b, kern):
2     f=0
3     for i in range(np.shape(w)[0]):
4         f+=w[i]*kern(imgs[nligne], imgs[i,:])
5     return f+b

```

3.4 Question 4

Afin de créer la matrice de confusion de notre programme pour le chiffre 0, nous créons une fonction qui nous permet d'obtenir la matrice de confusion, et qui nous calcule la sensibilité et le taux d'erreur de notre algorithme. (En supplément, de la matrice de confusion, en sortie, nous récupérons les lignes où le programme nous indique la bonne valeur, soit ici 0)

Création de la matrice de confusion

```

1 def mat_conf(imgs, W, b, nombredetection, label, kern):
2     Nvp = 0
3     Nfn = 0
4     Nfp = 0
5     Nvn = 0
6     N = len( label )
7     G = []
8     for k in range( 0 , N ) :
9
10         if fk(k, imgs, W, b, kern) > 0 and label [k] == nombredetection :
11             Nvp = Nvp + 1
12             G.append(k)
13         if fk(k, imgs, W, b, kern) <= 0 and label [k] == nombredetection :
14             Nfn = Nfn + 1
15         if fk(k, imgs, W, b, kern) > 0 and label [k] != nombredetection :
16             Nfp = Nfp + 1
17         if fk(k, imgs, W, b, kern) <= 0 and label [k] != nombredetection :
18             Nvn = Nvn + 1
19
20
21     t_xerreur = ( Nfp + Nfn ) / N
22     sensibilite = Nvp / ( Nvp + Nfn )
23     Mc = np. array ([[ Nvp , Nfn ] , [ Nfp , Nvn ] ])
24     print (f"Matrice de confusion pour {nombredetection} \n {Mc}")
25     print ('Taux d'erreur', t_xerreur )
26     print ('Sensibilit', sensibilite )
27
28     return Mc, G

```

3.5 Question 5

Afin de réaliser des algorithmes qui détectent les chiffres $i \in [1, 9]$ parmi les autres chiffres contenus dans la base de données MNIST en utilisant la méthode des SVM. Nous avons utilisé une boucle permettant

de remplir des grands W et B, que l'on remplit pour chaque chiffres, c'est pour cela que c'est array ont 10 colonnes.

Calcul des paramètre d'UZAWA pour les 10 chiffres

```
1 W=np.zeros((nbrdonnee,10))
2 B=np.zeros((1,10))
3
4 for i in range(10):
5     W[:,[i]], B[:,[i]] = Uzawak(i, kernpol, train_imgs, train_labels)
```

Afin d'obtenir la matrice de confusion pour tous les chiffres nous avons aussi réaliser une boucle.

Calcul de la matrice de confusion pour les 10 chiffres

```
1 for i in range(10):
2     Conf = mat_conf(test_imgs,W[:,i],B[0,i],i,test_labels,kernpol)[0]
```

Afin de comparer avec l'algorithme d'UZAWA sans noyau, il nous est nécessaire de rédiger cet algorithme sous la forme d'une fonction, avec la matrice de confusion intégré dans cette fonction

Fonction UZAWA sans noyau

```
1 def UZAWA(nombredetection, train_imgs, train_labels, test_labels, test_imgs):
2
3     indiceu =np. where ( train_labels == nombredetection )
4     indicev =np. where ( train_labels != nombredetection )
5     u= train_imgs [ indiceu ]
6     v= train_imgs [ indicev ]
7
8
9     p=np.shape(u)[0]
10    q=np.shape(v)[0]
11    n=np. shape (u) [1]
12
13
14    X=np. concatenate (( -1*u, v) ,axis =0)
15    e=np. block ([[ np. ones ((p ,1) ) ] , [ -np. ones ((q ,1) )]])
16    C=np. block ([X,e])
17
18    mu=np. ones (( np. shape (C) [0] ,1) )
19    maxiter=5000
20    epsilon=10**-2
21    cvmax =2 /( np. linalg . norm (C, ord =2) **2)
22    rho = cvmax /2
23    nb=0
24    mu0=mu+epsilon
25
26    while np.linalg.norm(mu-mu0) > epsilon and nb < maxiter :
27        x=(-1)*(C.T @ mu)
28        mu= proj (mu+ rho *( C@x +np. ones (( np. shape (C)[0] ,1) ) ) )
29        mu0=mu
30        nb+=1
31
32    w=np. zeros (n)
33    for i in range (0 ,p) :
34        w=w+mu[i]*u[i ,:]
35    for j in range (0 ,q) :
36        w=w-mu[p+j]*v[j ,:]
```

```

37
38
39 mu1 =mu [:p]
40 mu2 =mu[p:]
41 result1 =np. nonzero (mu1 ) [0]
42 result2 =np. nonzero (mu2 ) [0]
43 b=0
44 if result1.size >0 :
45     for i in np. nditer ( result1 ) :
46         b =(1/(2* len ( result1 ) ) ) *C[i ,:-1] @w +b
47
48 if result2.size >0 :
49     for j in np. nditer ( result2 ) :
50         b =(1/(2* len ( result2 ) ) ) *C[p+j ,:-1] @w+b
51
52
53
54
55 Nvp =0
56 Nfn =0
57 Nfp =0
58 Nvn =0
59
60 valeurs_test = test_labels
61 N= len( valeurs_test )
62
63 for k in range (0 ,N) :
64
65     if w.T @ test_imgs [k] -b >0 and valeurs_test [k ]== nombredetection :
66         Nvp =Nvp +1
67     if w.T @ test_imgs [k] -b <=0 and valeurs_test [k ]== nombredetection :
68         Nfn =Nfn +1
69     if w.T @ test_imgs [k] -b >0 and valeurs_test [k ]!= nombredetection :
70         Nfp =Nfp +1
71     if w.T @ test_imgs [k] -b <=0 and valeurs_test [k ]!= nombredetection :
72         Nvn =Nvn +1
73
74 txerreur =( Nfp +Nfn ) /N
75 sensibilite =Nvp /( Nvp + Nfn )
76 Mc=np. array ([[ Nvp , Nfn ] ,[Nfp , Nvn ] ])
77 print (f"Matrice de confusion pour s: {nombredetection}\n{Mc}")
78 print ( 'Taux d'erreur',txerreur )
79 print ( 'Sensibilit', sensibilite )

```

Voici les résultats que nous avons obtenue avec l'algorithme d'Uzawa avec noyau polynomial et sans noyau pour une marge de projection de 1 avec les 10 000 données de la base mnist-test :

nb	M_{conf} avec Uzawa sans noyau	T_e avec Uzawa sans noyau	T_s avec Uzawa sans noyau	M_{conf} avec Uzawa avec noyau poly- nomial	T_e avec Uzawa avec noyau poly- nomial	T_s avec Uzawa avec noyau poly- nomial
0	$\begin{pmatrix} 350 & 630 \\ 11 & 9009 \end{pmatrix}$	0.0641	0.3571	$\begin{pmatrix} 0 & 980 \\ 0 & 9020 \end{pmatrix}$	0.098	0.0
1	$\begin{pmatrix} 1063 & 72 \\ 347 & 8518 \end{pmatrix}$	0.0419	0.9366	$\begin{pmatrix} 0 & 1135 \\ 0 & 8865 \end{pmatrix}$	0.1135	0.0
2	$\begin{pmatrix} 0 & 1032 \\ 0 & 8968 \end{pmatrix}$	0.1032	0.0	$\begin{pmatrix} 0 & 1032 \\ 0 & 8968 \end{pmatrix}$	0.1032	0.0
3	$\begin{pmatrix} 0 & 1010 \\ 0 & 8990 \end{pmatrix}$	0.101	0.0	$\begin{pmatrix} 0 & 1010 \\ 0 & 8990 \end{pmatrix}$	0.101	0.0
4	$\begin{pmatrix} 62 & 920 \\ 4 & 9014 \end{pmatrix}$	0.0924	0.0631	$\begin{pmatrix} 0 & 982 \\ 0 & 9018 \end{pmatrix}$	0.0982	0.0
5	$\begin{pmatrix} 0 & 892 \\ 0 & 9108 \end{pmatrix}$	0.0892	0.0	$\begin{pmatrix} 0 & 892 \\ 0 & 9108 \end{pmatrix}$	0.0892	0.0
6	$\begin{pmatrix} 2 & 956 \\ 1 & 9041 \end{pmatrix}$	0.0957	0.0021	$\begin{pmatrix} 0 & 958 \\ 0 & 9042 \end{pmatrix}$	0.0958	0.0
7	$\begin{pmatrix} 331 & 697 \\ 3 & 8969 \end{pmatrix}$	0.07	0.3219	$\begin{pmatrix} 0 & 1028 \\ 0 & 8972 \end{pmatrix}$	0.1028	0.0
8	$\begin{pmatrix} 0 & 974 \\ 0 & 9026 \end{pmatrix}$	0.0974	0.0	$\begin{pmatrix} 0 & 974 \\ 0 & 9026 \end{pmatrix}$	0.0974	0.0
9	$\begin{pmatrix} 0 & 1009 \\ 0 & 8991 \end{pmatrix}$	0.1009	0.0	$\begin{pmatrix} 0 & 1009 \\ 0 & 8991 \end{pmatrix}$	0.1009	0.0

On test maintenant le même algorithme avec le même nombre de données mais avec cette fois ci avec une marge de projection de 10^5 :

nb	M_{conf} avec Uzawa sans noyau	T_e avec Uzawa sans noyau	T_s avec Uzawa sans noyau	M_{conf} avec Uzawa avec noyau poly- nomial	T_e avec Uzawa avec noyau poly- nomial	T_s avec Uzawa avec noyau poly- nomial
0	$\begin{pmatrix} 980 & 0 \\ 8156 & 864 \end{pmatrix}$	0.8156	1.0	$\begin{pmatrix} 0 & 980 \\ 0 & 9020 \end{pmatrix}$	0.098	0.0
1	$\begin{pmatrix} 1135 & 0 \\ 4703 & 4162 \end{pmatrix}$	0.4703	1.0	$\begin{pmatrix} 0 & 1135 \\ 3 & 8862 \end{pmatrix}$	0.1138	0.0
2	$\begin{pmatrix} 1023 & 9 \\ 5475 & 3493 \end{pmatrix}$	0.5484	0.9913	$\begin{pmatrix} 0 & 1032 \\ 0 & 8968 \end{pmatrix}$	0.1032	0.0
3	$\begin{pmatrix} 1002 & 8 \\ 6405 & 2585 \end{pmatrix}$	0.6413	0.9921	$\begin{pmatrix} 0 & 1010 \\ 0 & 8990 \end{pmatrix}$	0.101	0.0
4	$\begin{pmatrix} 980 & 2 \\ 5095 & 3923 \end{pmatrix}$	0.5097	0.9979	$\begin{pmatrix} 0 & 982 \\ 0 & 9018 \end{pmatrix}$	0.0982	0.0
5	$\begin{pmatrix} 805 & 87 \\ 2669 & 6439 \end{pmatrix}$	0.2756	0.9025	$\begin{pmatrix} 0 & 892 \\ 0 & 9108 \end{pmatrix}$	0.0892	0.0
6	$\begin{pmatrix} 954 & 4 \\ 6696 & 2346 \end{pmatrix}$	0.67	0.9958	$\begin{pmatrix} 0 & 958 \\ 0 & 9042 \end{pmatrix}$	0.0958	0.0
7	$\begin{pmatrix} 1022 & 6 \\ 6406 & 2566 \end{pmatrix}$	0.6412	0.9941	$\begin{pmatrix} 0 & 1028 \\ 0 & 8972 \end{pmatrix}$	0.1028	0.0
8	$\begin{pmatrix} 880 & 94 \\ 4910 & 4116 \end{pmatrix}$	0.5004	0.9034	$\begin{pmatrix} 0 & 974 \\ 0 & 9026 \end{pmatrix}$	0.0974	0.0
9	$\begin{pmatrix} 984 & 25 \\ 3763 & 5228 \end{pmatrix}$	0.3788	0.9752	$\begin{pmatrix} 0 & 1009 \\ 0 & 8991 \end{pmatrix}$	0.1009	0.0

3.6 Question 6

Afin d'obtenir une prédiction plus précise, nous calculons pour une ligne choisi, dans le cas où pour plusieurs chiffres, les réponses sont positives . Ainsi pour cela, on prendra la réponse la plus haute, qui correspondra à un certains chiffre, qui sera le chiffre prédit.

Fonction de prédiction

```

1 def pred(nligne : int , kern : callable , imgs : np.ndarray, labels : np.
  ndarray,W: np.ndarray,B : np.ndarray,plot : bool =False) -> int:
2     P=np.zeros(10)
3     for i in range(10):
4         P[i]=fk(nligne,imgs,W[:,i],B[:,i][0],kern)
5
6     if plot==True :
7         D=np.zeros(10)
8         minP=np.min(P)
9         for i in range(len(P)):
10            D[i]=np.linalg.norm(minP-P[i])
11
12         plt.figure()
13         plt.bar(np.arange(10),D/sum(D))
14
15     return np.argmax(P)

```

Afin d'obtenir la matrice de confusion et le taux de reconnaissance de ce programme, on crée une fonction spécifique, rangeant dant un tableau de taille 10 x 10, les différentes prédictions. Sachant que la diagonale correspond à une correspondance correct entre la prédiction et la vrai valeur.

Calcul du taux de reconnaissance avec la prédiction

```

1 def conf_gen(imgs: np.ndarray, W: np.ndarray, B: np.ndarray, labels: np.ndarray
  , kern: callable) -> np.ndarray:
2
3     ligne,col=np.shape(imgs)
4     Conf=np.zeros((10,10)).astype('int')
5
6     for i in range(ligne) :
7
8         lx=labels[i]
9         c=pred(i, kern, imgs, labels, W, B)
10        Conf[lx,c]+=1
11
12
13    print(Conf)
14    print("taux de réussite total: ", np.trace(Conf)/ligne)
15
16    return Conf

```

On a ainsi la matrice de confusion suivante, avec le noyau polynomiale et une marge de projection de 10^5 :

$$\begin{pmatrix}
 0 & 980 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1135 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1032 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1010 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 982 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 892 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 958 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1028 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 974 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
 0 & 1009 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
 \end{pmatrix}$$

Avec un taux de réussite total: 0.1135 soit 11.35% de réussite. Ce résultat s'explique car il faut jouer avec le facteur de marge ainsi que le coefficient C que l'on retrouve dans le noyau polynomial, mais aussi avec le nombre de données d'entraînement.

3.7 Question 7

Dans un premier temps, nous implémentons dans notre programme la fonction noyau gaussienne.

Noyau Gaussien

```

1 def kerngauss(x : np.ndarray ,y : np.ndarray, c : int =1):
2     return np.exp(-(np.linalg.norm(x-y)**2)/(2*c**2))

```

Par faute de temps et de puissance de calcul, nous testerons différents facteurs et noyaux sur 1000 données.

Nous obtenons alors les résultats suivant avec ce même noyau :

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 85 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 126 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 116 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 107 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 110 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 87 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 87 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 99 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 89 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 94 & 0 & 0 \end{pmatrix}$$

Avec un taux de réussite total: 0.099 soit 9.9% de réussite.

Ce taux de réussite semble faible mais nous rappelons que nous avons tester notre algorithme sur seulement 1000 données, ce qui limite grandement la précision de nos résultats.

3.8 Question 8

Nous allons maintenant dans cette question tester notre algorithme avec la base de données fashion. Cette dernière regroupe près de 10 000 données sur des vêtements vendu sur le site de zalando.

Avec notre algorithme, nous obtenons avec le noyau gaussien :

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 98 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 89 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 111 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 104 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 96 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 96 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 100 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 102 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 98 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 105 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Avec un taux de réussite total: 0.0961 soit 9.61% de réussite.

Ce faible taux s'explique par le gaussien qui n'est pas assez optimisé, entraîné et testé avec seulement 1000 données.

Afin de conclure ce rapport, nous souhaitons tout d'abord remercier nos professeurs de mathématiques M. EL MAHBOUBY et M. COUFFIGNAL pour le temps qu'ils ont consacré à nous aider dans la compréhension de ce BE. Ce bureau d'étude a été pour nous l'occasion de travailler en équipe. Nous avons fait face tous ensemble à plusieurs difficultés que nous avons su surmonter.

Une des premières difficulté majeure que nous avons rencontrée a été d'arriver à faire en sorte que les fonctions qui trace les fractales reconnaissent les différents cas de convergences.

Nous avons également eu des difficultés à comprendre le fonctionnement de l'algorithme d'Uzawa.

Finalement, ce projet nous a permis de redécouvrir plus en profondeur le fonctionnement des fractales mais également celui de la fonction SVM avec noyau ainsi que leurs nombreuses propriétés et applications avec l'algorithme d'Uzawa par exemple. Mais il nous a également permis de consolider nos connaissances sur ce module d'optimisation différentiable.

Comme ouverture à ce projet, principalement autour de l'utilisation de l'algorithme d'UZAWA avec noyau, avec une préférence pour le noyau gaussien. On pourrait mettre en place un système utilisant la librairie tkinter afin de créer un paint permettant de rajouter de nouvelle données d'entraînement, ainsi on pourrait cliquer sur valider, une fois un chiffre dessiner et avoir dans le même temps la prédiction du chiffre dessiner.